

SMART CAMPUS EVENT MANAGEMENT SYSTEM

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

E.Mani Venkata Avinash(VTU25681)

10211CS224

Full Stack Application Development

Slot :E1-B4



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "SMART CAMPUS EVENT MANAGEMENT SYSTEM " by E.Mani Venkata Avinash (VTU25681) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr.S.Hemamalini

Assistant Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr.Sumithra.M

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(E.Mani Venkata Avinash)

Date: 06/05/2026

ABSTRACT

The Smart Campus Event Management Application is a full stack web-based system designed to streamline the organization, registration, and management of campus events. Traditional event handling methods often rely on manual processes, leading to inefficiencies, data loss, and poor user experience.

The application enables users to browse events, select ticket types, and complete registrations through an interactive and user-friendly interface. It incorporates dynamic frontend technologies for seamless navigation and backend services for secure data handling and storage. The system ensures accurate record keeping by storing participant details, ticket information, and booking confirmations in a structured database. It also enhances user engagement by offering instant confirmations and a structured booking experience. Overall, the project demonstrates how modern web technologies can be leveraged to build scalable, efficient, and user-centric solutions for campus event management.

Keywords: Event Management, Full Stack Development, Web Application, Ticket Booking System, Database Management, Campus Automation.

LIST OF FIGURES

1.1	Framework Diagram	4
3.1	Structure of the project	10
3.2	Schema Structure Diagram	12
4.1	Proposed System Architecture of Smart Campus Event Management Application	15
4.2	Data Flow Diagram	16
4.3	Data Flow Diagram	18
5.1	Login Page	21
5.2	Admin Dashboard	22

LIST OF TABLES

2.1	Comparison of Existing Event Management Applications	7
7.1	SDG Mapping for the Project	25
7.2	Mapping of Project to Complex Engineering Activities (CEA)	26
7.3	Mapping of Project to Complex Engineering Problems (CEP)	27

LIST OF ACRONYMS AND ABBREVIATIONS

CI	Continuous Integration
ORM	Object Relational Mapping
UI	User Interface
UX	User Experience
API	Application Programming Interface
DBMS	Database Management System
SQL	Structured Query Language
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
HTTP	HyperText Transfer Protocol
CRUD	Create, Read, Update, Delete

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	2
1.3 Scope of the Project	2
1.4 Framework	3
2 SURVEY OF SIMILAR APPLICATION IN MARKET	5
3 FRAMEWORK DESCRIPTION	8
3.1 Frontend Implementation	8
3.1.1 Environment Setup	8
3.1.2 Structure of the Project	8
3.1.3 Execution Procedure	9

3.2	Backend Implementation	9
3.2.1	Environment Setup	9
3.2.2	Structure of the Project	10
3.2.3	Execution Procedure	11
3.3	Database Implementation	11
3.3.1	Database Configuration	11
3.3.2	Schema Structure	12
3.3.3	Tables Created	12
4	METHODOLOGIES	14
4.1	Project Architecture	14
4.2	DataFlow Diagram	16
4.3	Module 1: User Interface Module	17
4.4	Module 2: API and Backend Module	17
4.5	Module 3: Database Module	17
4.6	Advantages of this Project	18
4.7	Disadvantages	19
5	RESULTS AND DISCUSSIONS	20
5.1	System Results	20
5.2	Discussion	21
6	CONCLUSION AND FUTURE ENHANCEMENTS	23

6.1	Conclusion	23
6.2	Future Enhancements	24
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	25
7.1	SDG Mapping	25
7.2	Mapping of the Project to Complex Engineering Ac- tivities (CEA)	26
7.3	Mapping of Project to Complex Engineering Problems (CEP)	27
8	SOURCE CODE	28
8.1	Source Code	28
	REFERENCES	33

Chapter 1

INTRODUCTION

1.1 Introduction

Managing campus events through manual or semi-digital methods often leads to inefficiencies, lack of coordination, and poor user experience. To overcome these challenges, a Smart Campus Event Management Application is developed as a full stack web-based system that centralizes event management and ticket booking processes.

The application enables users to explore events, select ticket types, and register seamlessly through an interactive interface. It also provides organizers with a structured platform to manage event data, participant records, and bookings in real time. By integrating frontend and backend technologies, the system ensures efficient data flow, improved accessibility, and enhanced user engagement, making the entire event management process faster, reliable, and scalable.

1.2 Aim of the Project

The aim of this project is to design and develop a full stack web application for efficient management of campus events and ticket bookings. The system focuses on simplifying event registration, improving user interaction, and ensuring accurate data handling through a centralized platform. It also aims to reduce manual effort, enhance accessibility, and provide a seamless experience for both users and event organizers.

1.3 Scope of the Project

The scope of this project includes the development of a web-based application for managing campus events such as seminars, workshops, hackathons, and technical fests. The system allows users to view event details, select ticket types, and complete registrations, while enabling organizers to store and manage booking data efficiently.

The application emphasizes a responsive user interface, real-time interaction, and reliable data storage. Future enhancements may include integration of online payment systems, QR code-based ticket verification, email notifications, and analytics dashboards for monitoring event performance and user engagement.

1.4 Framework

The proposed Smart Campus Event Management Application is developed using a full stack architecture combining frontend and backend technologies. The frontend is built using HTML, CSS, and JavaScript to create a responsive and user-friendly interface for event browsing and ticket booking.

The backend is developed using Node.js with Express.js, which handles server-side logic, API creation, and data processing. MySQL is used as the database management system to store and manage user details, event information, and booking records efficiently.

This architecture ensures smooth communication between the client and server, enabling real-time data handling, scalability, and efficient performance. The modular design allows easy expansion and integration of additional features, making the system adaptable for future improvements.

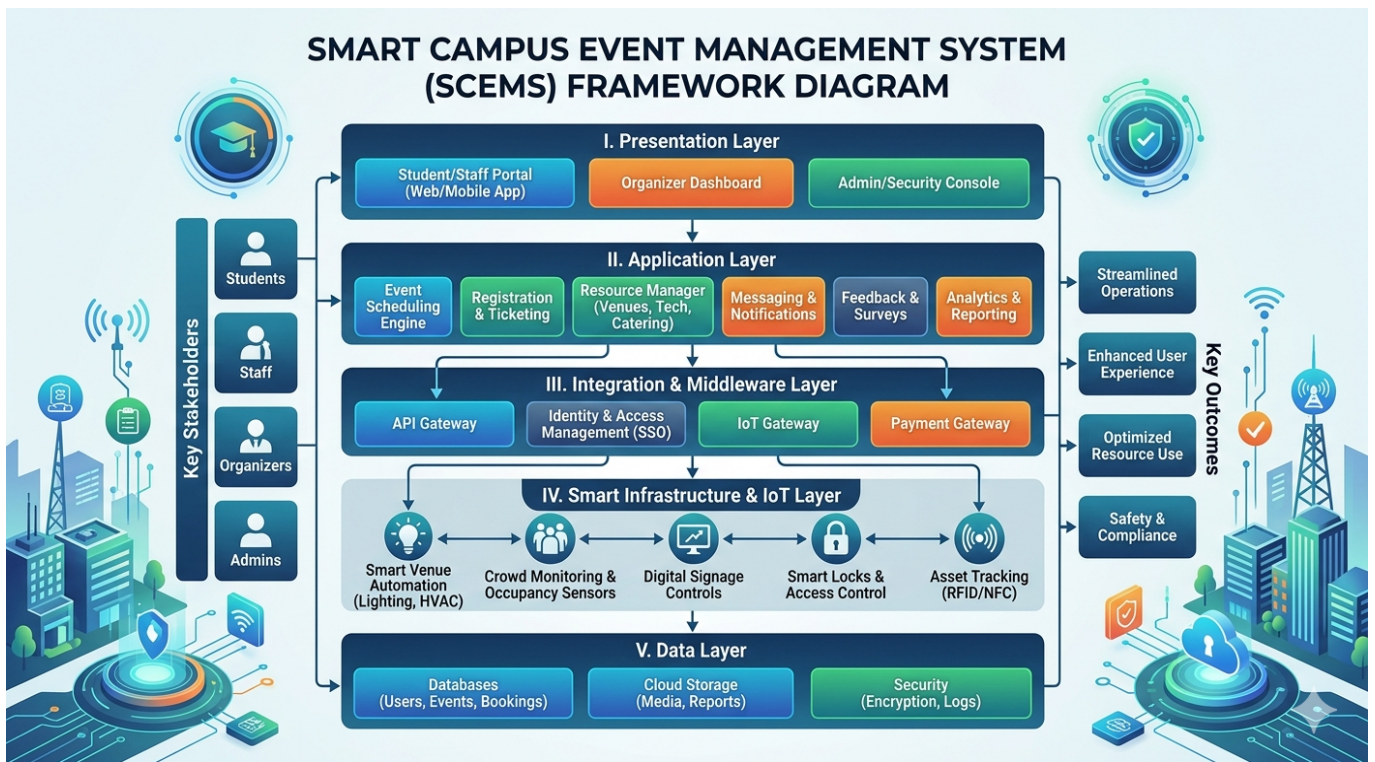


Figure 1.1: Framework Diagram

Fig 1.1 illustrates the framework of the Smart Campus Event Management System, showcasing the structured multi-layered architecture. At the top level, users interact with the frontend views (Thymeleaf and Leaflet map). These interactions trigger HTTP requests to the Spring Boot controllers, which interact with the service layer to process logic, eventually interacting with the MySQL database through Spring Data JPA repositories.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

In the current market, several online event management and ticket booking platforms have been developed to simplify event discovery, registration, and reservation processes. One of the most widely used platforms in India is BookMyShow, which provides services for booking tickets for movies, concerts, sports events, and live shows. It offers features such as event listings, seat selection, secure payment integration, and a user-friendly interface, making it highly popular among users [ref1].

Similarly, global platforms like Ticketmaster and StubHub provide large-scale ticketing solutions with advanced features such as event promotion, dynamic pricing, secure transactions, and customer support. These systems are designed to handle millions of users and large-scale events efficiently, ensuring reliability and scalability [ref2].

Most modern event management applications share common functionalities including user registration, event browsing, ticket booking, real-time availability updates, and booking history tracking. Advanced

platforms also include features such as personalized recommendations, analytics dashboards for organizers, QR-based ticket verification, and fraud detection mechanisms to enhance security and user experience [ref3].

Despite their advantages, existing systems often have certain limitations such as high service charges, complex user interfaces, dependency on online payments, and system overload during peak booking periods. Additionally, these platforms are generally designed for large-scale commercial events and may not be suitable for smaller environments like campus or departmental events.

This survey indicates that while current applications are powerful and feature-rich, there is a need for a simplified, cost-effective, and customizable solution tailored for academic institutions. The proposed Smart Campus Event Management Application addresses this gap by providing a lightweight, user-friendly, and efficient system specifically designed for managing campus events and registrations. The market for event management is segmented into specialized niches, ranging from high-end Enterprise Systems like Cvent to user-friendly Ticketing Platforms like Eventbrite. While these existing solutions offer robust features such as advanced analytics, global marketing, and scalability. A significant gap exists in providing a localized, visual-first experience for academic settings. Most mainstream applications, such as

Eventzilla or Eventify, focus heavily on the administrative side (ticketing and registration) but lack built-in tools for geographical navigation on a specific campus. Furthermore, systems that are campus-focused, such as Blackthorn, often have high barriers to entry, such as requiring integration with external platforms like Salesforce. us the comparison

Application	Type	Key Features	Advantages	Limitations
Whova	Conference App	Registration, Networking, Analytics	Interactive, Feature-rich	Complex for small use
Eventbrite	Ticketing Platform	Event creation, Ticket booking, Payments	Easy to use	Not campus-specific, no map
Cvent	Enterprise System	Venue mgmt, Marketing, Analytics	Scalable, powerful	Expensive
Eventify	Event Platform	Q&A, Registration, Dashboard	Real-time engagement	Limited customization
Eventzilla	Registration Tool	Ticketing, Custom pages	Flexible pricing	Focus on ticketing only
LineUp	Engagement App	Notifications, Schedules	Good user interaction	Limited management features
Blackthorn	University System	QR check-in, Analytics	Campus-focused	Requires Salesforce
SCEMS (Ours)	Campus System	Leaflet Map, Event Registration	Intuitive visual discovery	Restricted to Campus scope

Table 2.1: Comparison of Existing Event Management Applications

Table 2.1 shows existing event management applications. The market for event management is segmented into specialized niches, ranging from high-end Enterprise Systems like Cvent to user-friendly Ticketing Platforms like Eventbrite. While these existing solutions offer robust features such as advanced analytics, global marketing, and scalability.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend development environment is set up using HTML, CSS, and JavaScript for building a responsive and interactive user interface. The project is developed using Visual Studio Code as the code editor. Node.js is used to support package management if required, and testing is performed using modern web browsers such as Google Chrome. The frontend communicates with the backend using HTTP requests to handle booking operations and data exchange.

3.1.2 Structure of the Project

The frontend follows a structured and modular design for better maintainability. The application consists of multiple sections such as home page, event listing page, and ticket booking page. JavaScript functions handle dynamic behaviors like ticket selection, form validation, and navigation between pages. CSS is used for styling and layout design to ensure a clean and responsive interface. The code is orga-

nized into sections for UI components, styling, and scripts to maintain clarity.

3.1.3 Execution Procedure

To execute the frontend, the project files are opened in a web browser. The user can navigate through different sections such as viewing events and booking tickets. When a user submits the booking form, the frontend sends data to the backend server through API calls. The response from the backend is then displayed to the user in the form of booking confirmation.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend environment is configured using Node.js and Express.js to handle server-side logic and API requests. MySQL is used as the database system for storing booking information. The server is developed and tested using Visual Studio Code, and the API is accessed locally through a server running on a specified port (e.g., localhost:5000).

3.2.2 Structure of the Project

The backend follows an API-based architecture. It includes endpoints such as /book for handling ticket bookings and /bookings for retrieving stored data. The server processes incoming requests, validates user input, and interacts with the MySQL database to store booking records. The structure includes modules for server configuration, database connection, and API handling, ensuring separation of concerns. The server processes incoming requests, Figure 3.1 The

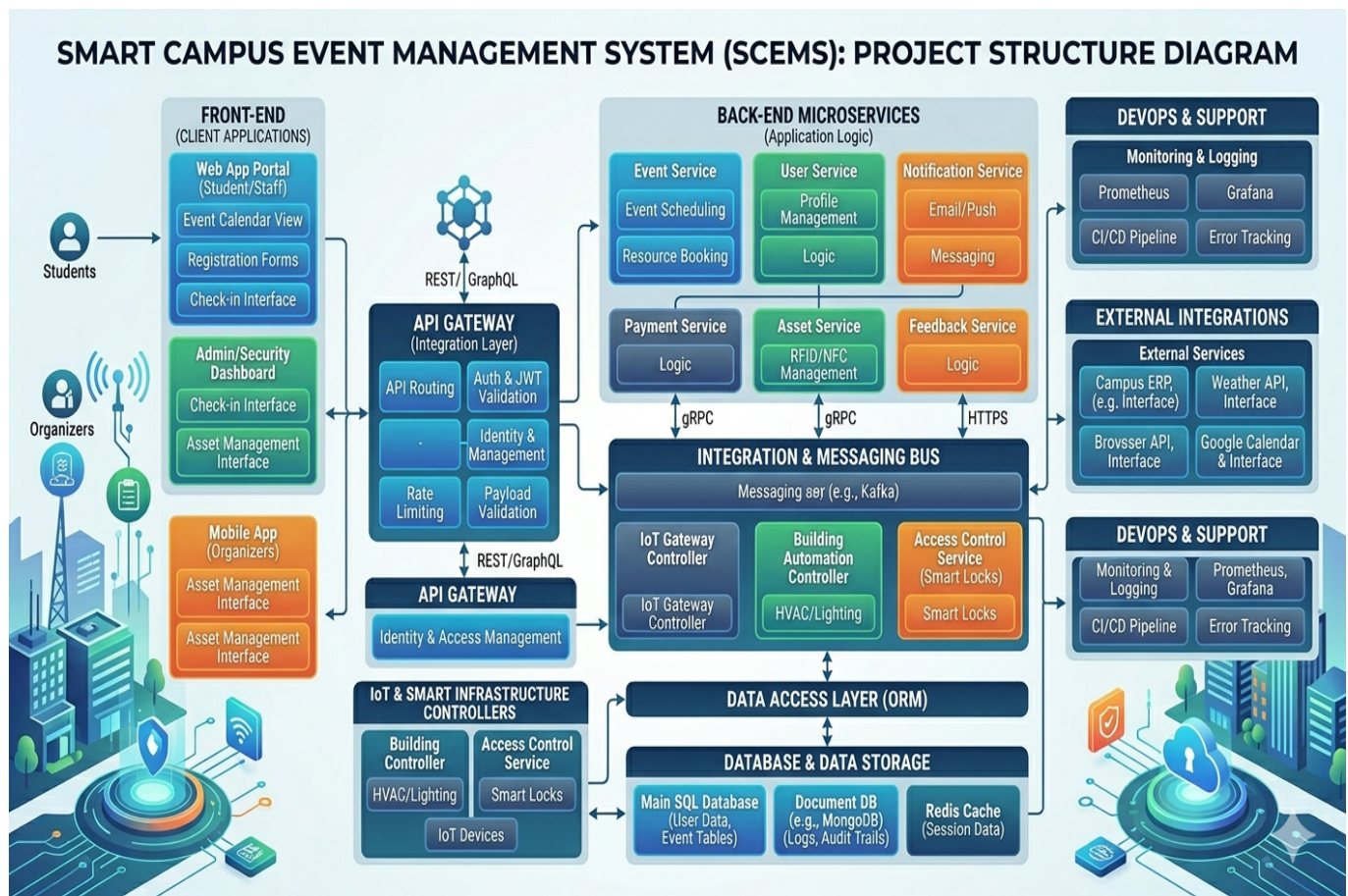


Figure 3.1: Structure of the project

(Figure 3.1) project illustrates a robust microservices architecture that

connects front-end client applications with a multifaceted back-end. It highlights a specialized integration of IoT and smart infrastructure controllers.

3.2.3 Execution Procedure

To run the backend, the MySQL database is first started and connected. The Node.js server is then executed using the command `node server.js`. Once the server is running, it listens for incoming requests from the frontend. When a booking request is received, the server processes the data and stores it in the database. The stored data can be verified using SQL queries such as `SELECT * FROM bookings;`

3.3 Database Implementation

3.3.1 Database Configuration

The database is configured using MySQL to store and manage booking data. A database named `events` is created and selected using SQL commands. The configuration is done through the MySQL command line interface, ensuring proper setup for data storage and retrieval.

3.3.2 Schema Structure

The database schema is designed to handle booking information efficiently. It includes structured fields such as name, email, department, ticket type, event name, quantity, amount, and booking ID. These fields ensure accurate storage and retrieval of user booking details.

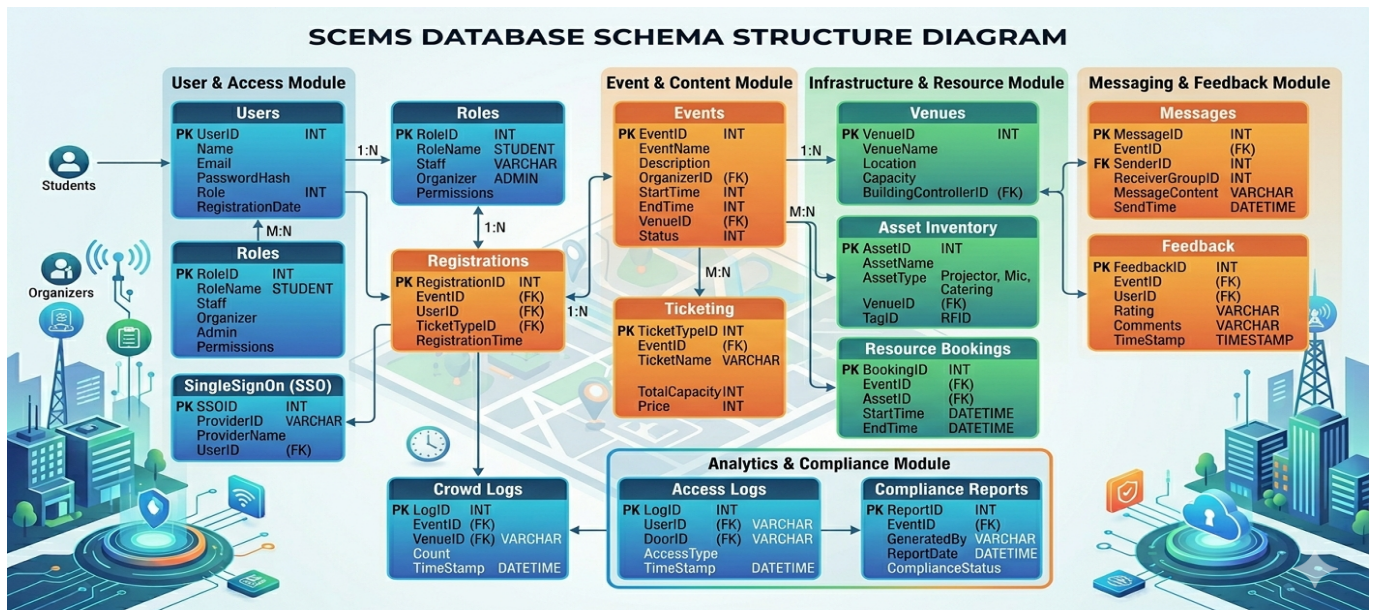


Figure 3.2: Schema Structure Diagram

Figure 3.2 showcases the structure of the schema for the project. The database is structured into five distinct functional modules: User Access, Event Content, Infrastructure Resource, Messaging Feedback, and Analytics Compliance, ensuring a scalable and organized data environment..

3.3.3 Tables Created

The core tables implemented in the system include:

- **events:** Stores comprehensive details about the event including `id`, `title`, `description`, `dateTime`, `location`, `latitude`, `longitude`, `department`, `capacity`, and `currentRegistrations`. The spatial coordinates allow for map plotting.
- **registrations:** Records student participation. It contains fields such as `id`, `event_id` (foreign key), `studentName`, `studentEmail`, `studentId`, `department`, and `registrationDate`.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

Figure 4.1 show cases the project follows a layered architecture consisting of presentation, application, and database layers. The presentation layer includes the frontend developed using HTML, CSS, and JavaScript, which provides an interactive interface for users to explore events and book tickets. The application layer is powered by a Node.js and Express backend that handles API requests, processes booking logic, and ensures smooth communication between the frontend and database. The database layer uses MySQL to store and manage event and booking records efficiently.

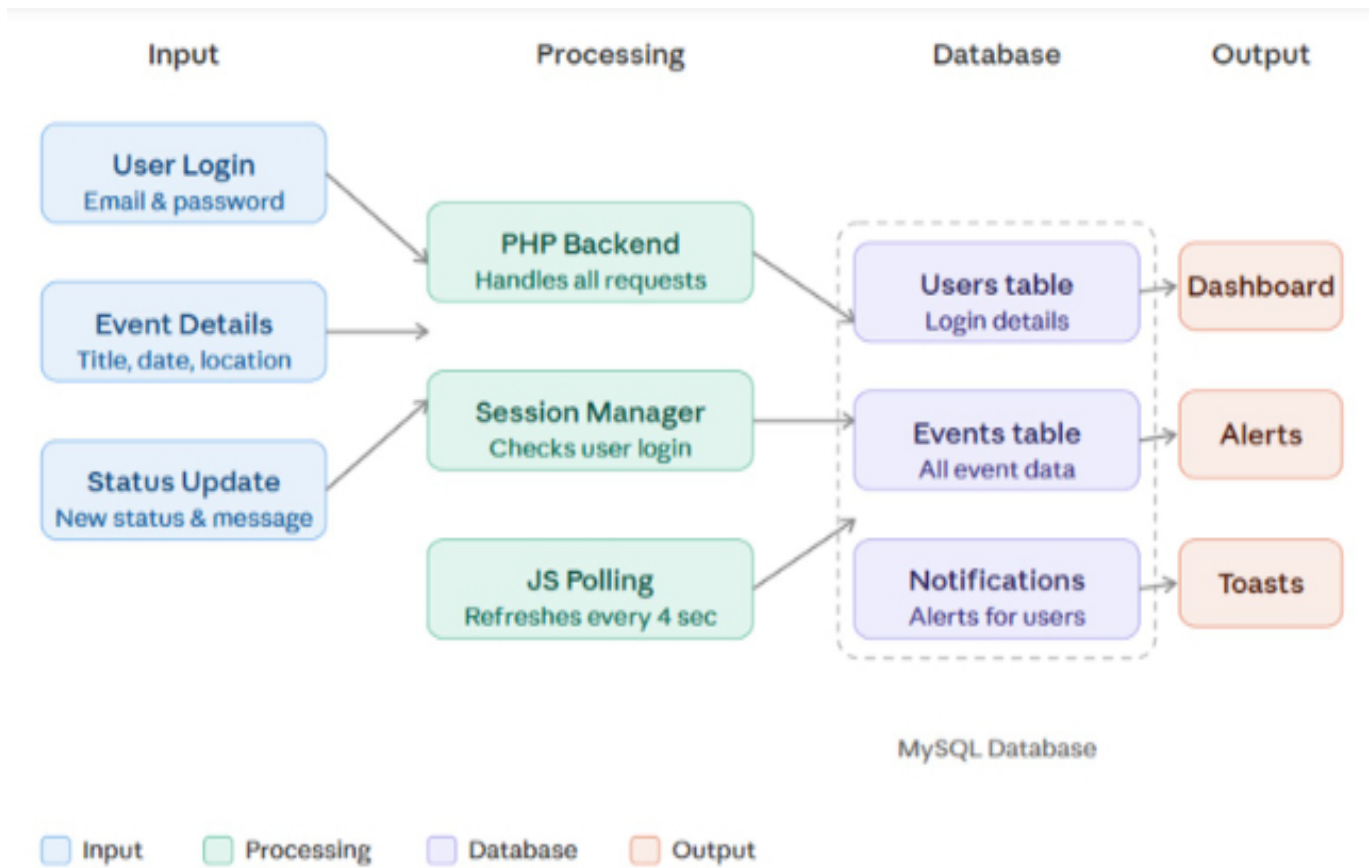


Figure 4.1: Proposed System Architecture of Smart Campus Event Management Application

The frontend communicates with the backend through RESTful APIs, while the backend interacts with the database to perform data storage and retrieval operations. This separation of concerns improves scalability, maintainability, and performance of the system. Future enhancements such as authentication, payment gateways, and cloud deployment can be integrated seamlessly into this architecture.

4.2 DataFlow Diagram

show us the data flow begins when the user accesses the web application and views the list of available events. The frontend sends a request to the backend API to fetch event data. When a user selects an event and submits the booking form, the entered details are validated on the client side and sent to the backend server.

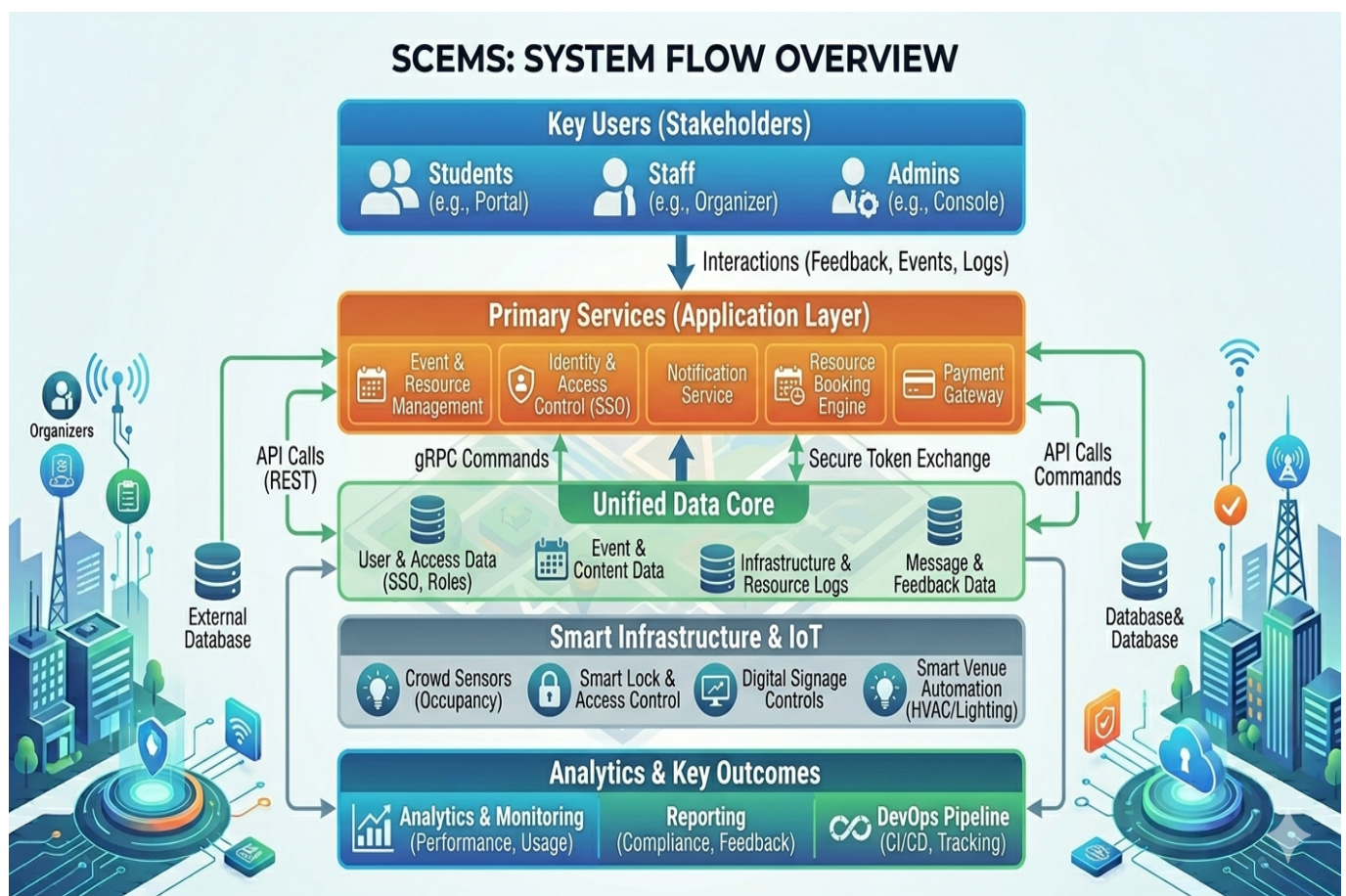


Figure 4.2: Data Flow Diagram

Figure 4.2 The backend processes the request, performs validation, and stores the booking information in the MySQL database. It also updates ticket availability if required. After successful storage, a con-

firmation response is sent back to the frontend, which displays booking confirmation details to the user.

4.3 Module 1: User Interface Module

This module is responsible for the design and interaction of the application. It includes event listings, filtering options, ticket selection, and the booking form. The module ensures a clean and user-friendly experience using structured layouts, responsive design, and interactive elements.

4.4 Module 2: API and Backend Module

This module manages communication between the frontend and database. It includes API endpoints for handling ticket bookings and registrations for retrieving booking records. The backend validates user inputs, processes business logic, and ensures reliable data transfer.

4.5 Module 3: Database Module

Figure 4.3 shows the module which handles data storage and management. It uses MySQL to maintain structured tables such as bookings, which store user details, ticket type, selected event, quantity, and

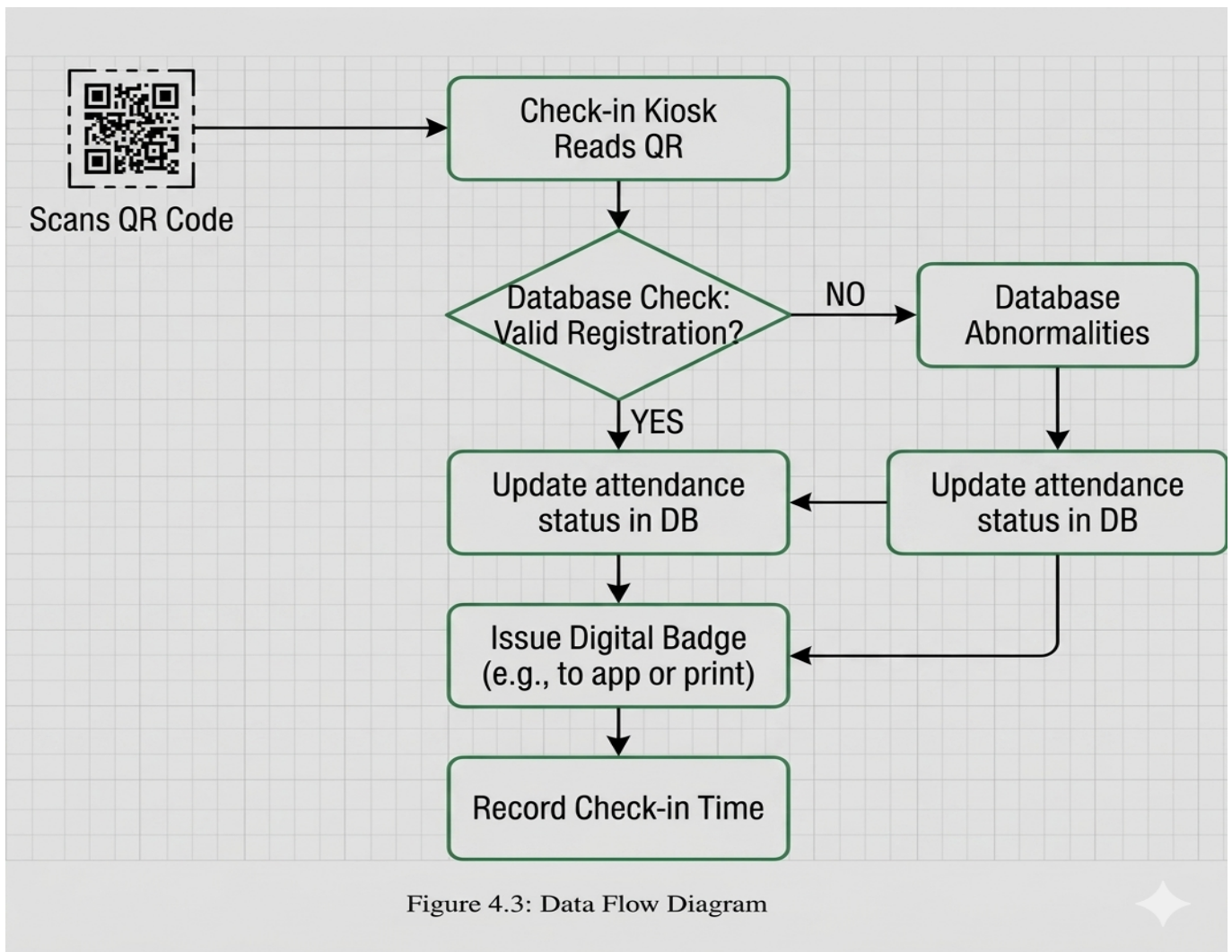


Figure 4.3: Data Flow Diagram

total amount. It ensures efficient data retrieval and consistency across the system.

4.6 Advantages of this Project

This project provides a streamlined and digital approach to managing campus event registrations and ticket bookings.

a) User-Friendly Interface: The system offers an intuitive and responsive interface, making it easy for users to browse events and com-

plete bookings quickly.

b) Real-Time Data Handling: The application dynamically updates booking information, ensuring accurate and up-to-date ticket availability.

c) Centralized Data Management: All booking and event details are stored in a structured database, simplifying data access and management.

4.7 Disadvantages

The system has certain limitations that can be addressed in future improvements. It currently depends on a local server setup, limiting accessibility beyond a specific environment. Advanced features such as secure authentication, online payment integration, and scalability for large-scale events are not yet implemented. Additionally, performance optimization may be required for handling high traffic scenarios.

Chapter 5

RESULTS AND DISCUSSIONS

5.1 System Results

The Smart Campus Event Management System was successfully developed using Spring Boot, Thymeleaf, and Leaflet.js. The results demonstrate that the system efficiently handles event data, correctly plots locations on the interactive map, and manages student registrations flawlessly while adhering to capacity constraints. The following screenshots illustrate the working of the system: The login page allows users to securely access the system using their credentials. The dashboard provides an overview of events and registration statistics. This page showcases the core innovation: the interactive map plotting event locations based on geographical coordinates, alongside the detailed list of events.

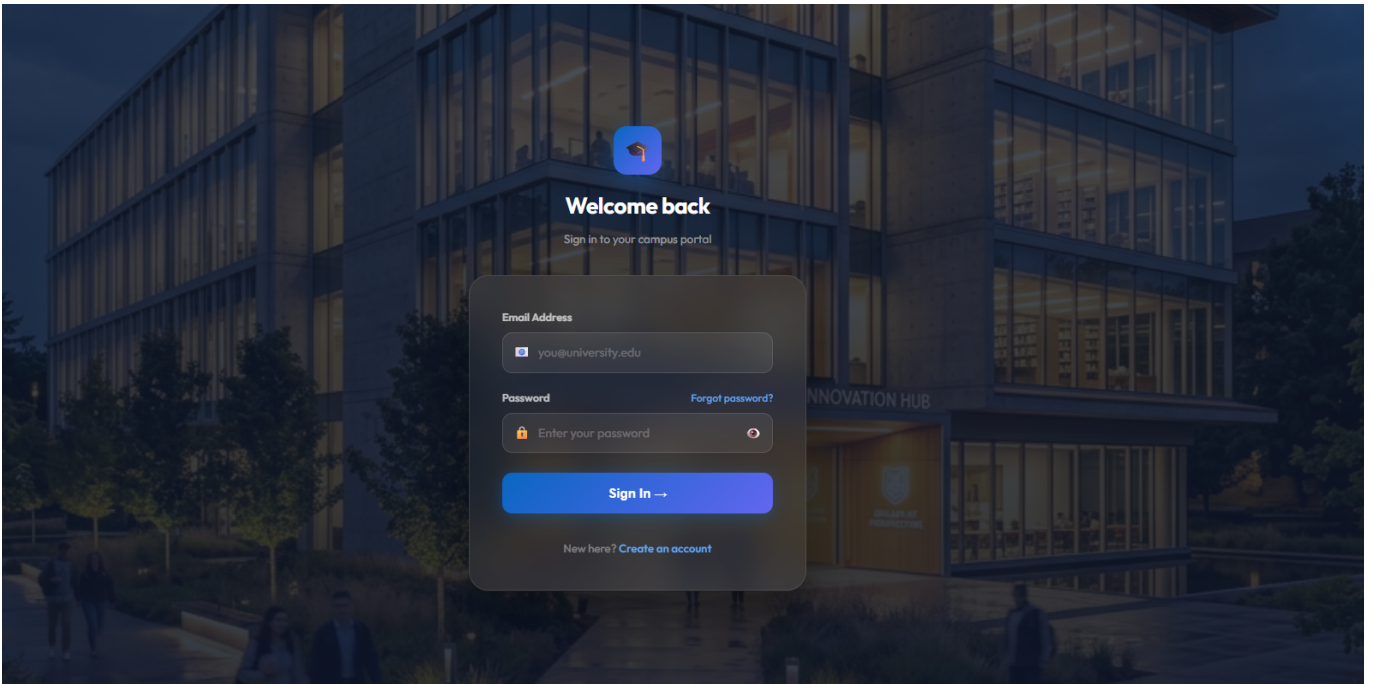


Figure 5.1: Login Page

Figure 5.1 shows the admin’s login interface .The results of the tests indicate that the system performs efficiently for small- to medium-scale usage, with accurate data storage and retrieval. The frontend interface is responsive and user-friendly, while the backend ensures proper validation and processing of booking data. Screenshots of the application interface, booking form, and database records are included below for reference.

5.2 Discussion

The results demonstrate that the proposed system effectively reduces manual work, improves booking accuracy, and provides a structured approach to event management. Compared to existing. Future

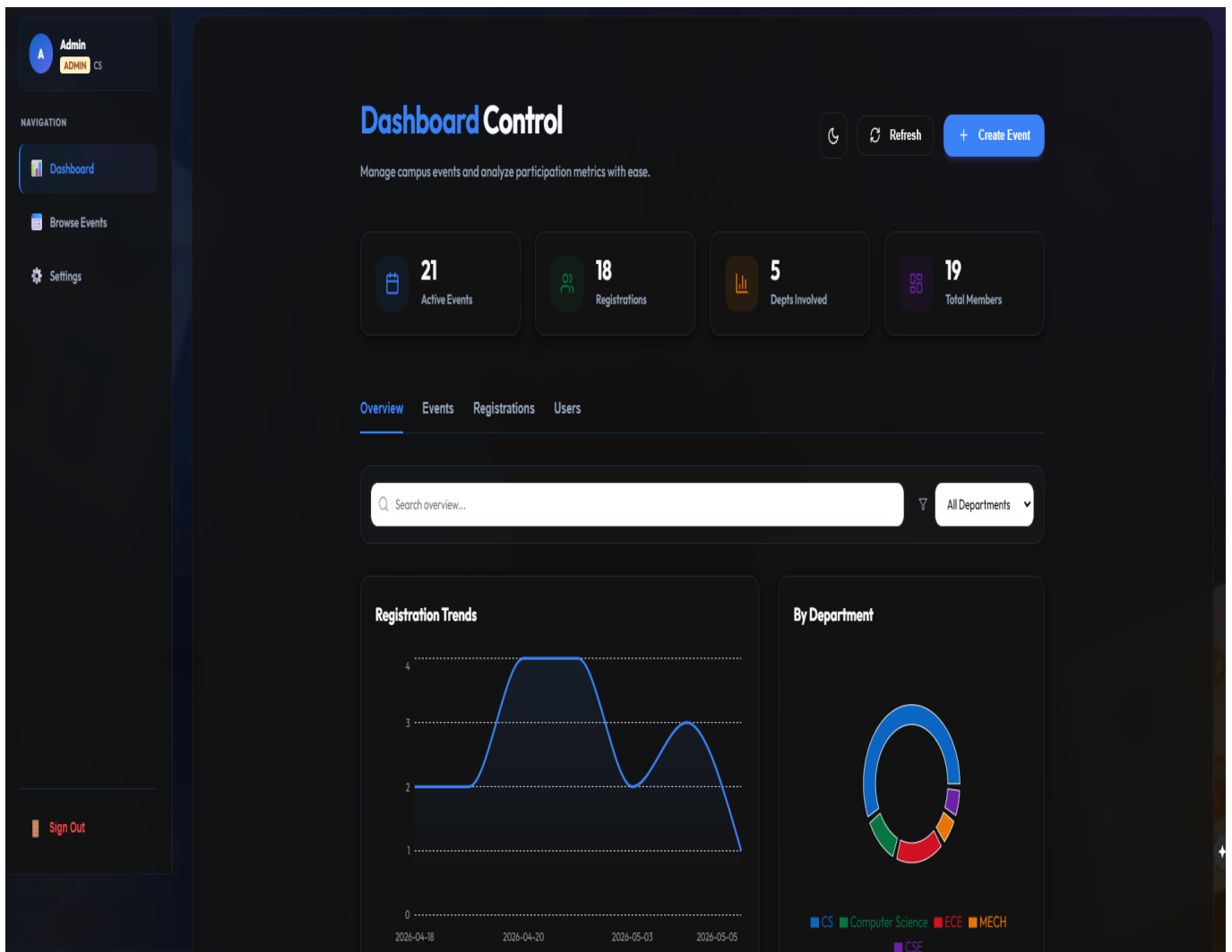


Figure 5.2: Admin Dashboard

Figure 5.2 improvements can further enhance performance, scalability, and feature integration such as online payments and authentication. Participants utilize this interface to submit their academic details to secure a spot for an event. Administrators configure new events here, including setting capacity limits and geographical coordinates. The relational database securely storing the entities. Figure 5.2 shows us the dashboard of the admin.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Smart Campus Event Management Application was successfully designed and implemented to digitize and simplify event registration and ticket booking within an academic environment. The system integrates a responsive frontend built using HTML, CSS, and JavaScript with a Node.js and Express backend, along with a MySQL database for efficient data storage.

The application enables users to browse events, select ticket types, and complete bookings through an intuitive interface. It ensures proper validation of user inputs, accurate calculation of ticket amounts, and reliable storage of booking details including event selection. The seamless communication between frontend and backend through API requests ensures real-time data handling and smooth user experience.

Compared to existing large-scale platforms, the proposed system is lightweight, cost-effective, and specifically tailored for campus-level

events. It reduces manual effort, minimizes errors, and provides a structured approach to managing event registrations efficiently.

6.2 Future Enhancements

The current system lays a strong foundation but can be significantly enhanced to create a more powerful and scalable platform. Future improvements include:

- Implementing user authentication and login system for secure and personalized access
- Integrating online payment gateways for real-time ticket booking and transactions
- Adding email and SMS notifications for instant booking confirmation and reminders
- Development of an admin dashboard for managing events, tracking bookings, and analytics
- Implementing QR code-based e-ticket verification for faster check-in
- Deploying the application on cloud platforms for high availability and scalability

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 SDG Mapping

Table 7.1: SDG Mapping for the Project

SDG No.	SDG Title	Relevance to the Project
SDG 9	Industry, Innovation and Infrastructure	Promotes digital infrastructure and automation in campus systems
SDG 11	Sustainable Cities and Communities	Supports smart campus ecosystem through efficient event management
SDG 12	Responsible Consumption and Production	Reduces paper usage by enabling digital ticket booking
SDG 16	Peace, Justice and Strong Institutions	Ensures secure data handling and reliable system operations

Table 7.1 shows us the SDG Mapping of this project contributes to Industry, Innovation and Infrastructure by promoting the development of digital infrastructure and automation within campus systems.

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of diverse technologies	HTML, CSS, JavaScript, Node.js, Express, and MySQL used together
EA2	Level of Interactions	Complex module interactions	API-based communication between frontend, backend, and database
EA3	Innovation	Modern technologies	Real-time booking system with dynamic UI and database updates
EA4	Societal Impact	Impact on users	Improves efficiency of campus event management and reduces manual work
EA5	Familiarity	Beyond standard practices	Full stack system integrating multiple layers and real-time processing

Table 7.2 shows the mapping of Project to Complex Engineering Activities (CEA) The project demonstrates a broad range of resources by integrating a diverse full-stack technology suite, including HTML, CSS, and JavaScript for the frontend, powered by Node.js and Express on the backend, and MySQL for data management.

7.3 Mapping of Project to Complex Engineering Problems (CEP)

Table 7.3: Mapping of Project to Complex Engineering Problems (CEP)

Level	Attribute	Characteristics	Justification
WP1	Depth of Knowledge	Requires engineering knowledge across multiple domains	Full stack development involving frontend, backend, and database technologies
WP2	Conflicting Requirements	Technical and usability trade-offs	Balancing performance, user experience, and data consistency
WP3	Depth of Analysis	Analytical and design thinking	Designing APIs, database schema, and system workflows
WP4	Familiarity	Partially new problem domain	Customized solution for campus-specific event management
WP5	Applicable Codes	Limited standardization	Custom booking logic without strict external standards
WP6	Stakeholder Needs	Multiple stakeholders involved	Students, organizers, and administrators interacting with system
WP7	Interdependence	Integration across multiple layers	Strong dependency between frontend, backend, and database

Table 7.3 shows the mapping of Project to Complex Engineering Problems (CEP)

Chapter 8

SOURCE CODE

8.1 Source Code

```
const nodemailer = require("nodemailer");

// Gmail transporter      uses App Password (not your real password)
const transporter = nodemailer.createTransport({
  service: "gmail",
  auth: {
    user: process.env.GMAIL_USER || "medinaveen05@gmail.com",
    pass: process.env.GMAIL_APP_PASSWORD || "",
  },
});

async function sendOTPEmail(toEmail, otp) {
  const mailOptions = {
    from: "EventHub■Security" <${process.env.GMAIL_USER || "medinaveen05@gmail.com"}>',
    to: toEmail,
    subject: "EventHub■Admin■Login■      ■Your■OTP■Code",
    html: `
      <div style="font-family:■Arial,■sans-serif;■max-width:■480px;■margin:■0■auto;■padding:■10px 0 0 0;">
        <div style="text-align:■center;■margin-bottom:■24px;">
          <h1 style="color:■#1a1a2e;■font-size:■24px;■margin:■0;">EventHub </h1>
          <p style="color:■#6b7280;■margin:■4px■0■0;">Admin Login Verification </p>
        </div>

        <div style="background:■white;■border-radius:■8px;■padding:■24px;■text-align:■center;">
          <p style="color:■#374151;■margin:■0■0■16px;">Your one-time verification code is:
          <div style="font-size:■40px;■font-weight:■bold;■letter-spacing:■12px;■color:■#4f4f4f;">
            ${otp}
          </div>
          <p style="color:■#6b7280;■font-size:■14px;■margin:■0;">
```

```

        This code expires in <strong>10 minutes</strong>.<br/>
        Do not share this code with anyone.
    </p>
</div>

<p style="color: #9ca3af; font-size: 12px; text-align: center; margin-top: 24px;">
    If you did not request this, please ignore this email.
</p>
</div>
',
};

await transporter.sendMail(mailOptions);
}

module.exports = { sendOTPEmail, sendBookingAcceptedEmail };

async function sendBookingAcceptedEmail(booking, attendees) {
    const adminEmail = process.env.GMAIL_USER || "medinaveen05@gmail.com";

    const attendeeRows = attendees.map(
        (a) => `<tr>
            <td style="padding: 8px 12px; border-bottom: 1px solid #e5e7eb;">${a.attendee_no}</td>
            <td style="padding: 8px 12px; border-bottom: 1px solid #e5e7eb;">${a.name}</td>
        </tr>`
    ).join("");

    const mailOptions = {
        from: "EventHub Admin <${adminEmail}>",
        to: adminEmail,
        subject: `Booking Accepted ${booking.event_name} (#${booking.id})`,
        html: `
            <div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; padding: 32px;">
                <div style="text-align: center; margin-bottom: 24px;">
                    <h1 style="color: #1a1a2e; font-size: 24px; margin: 0;">EventHub</h1>
                    <p style="color: #6b7280; margin: 4px 0 0 0;">Booking Accepted Admin Summary</p>
                </div>

                <div style="background: white; border-radius: 8px; padding: 24px; border: 1px solid #e5e7

```

```

<h2 style="color:#4f46e5;margin:0 0 16px;font-size:18px;"> Booking #${bookingId}

<table style="width:100%;border-collapse: collapse;font-size:14px;">
  <tr><td style="padding:6px 0;color:#6b7280;width:140px;">Event</td><td style="padding:6px 0;color:#6b7280;">
  <tr><td style="padding:6px 0;color:#6b7280;">Department</td><td style="padding:6px 0;color:#6b7280;">
  <tr><td style="padding:6px 0;color:#6b7280;">Date</td><td style="padding:6px 0;color:#6b7280;">
  <tr><td style="padding:6px 0;color:#6b7280;">Venue</td><td style="padding:6px 0;color:#6b7280;">
  <tr><td style="padding:6px 0;color:#6b7280;">Booked By</td><td style="padding:6px 0;color:#6b7280;">
  <tr><td style="padding:6px 0;color:#6b7280;">Tickets</td><td style="padding:6px 0;color:#6b7280;">
  <tr><td style="padding:6px 0;color:#6b7280;">Total Amount</td><td style="padding:6px 0;color:#6b7280;">
  <tr><td style="padding:6px 0;color:#6b7280;">Booked On</td><td style="padding:6px 0;color:#6b7280;">
</table>
</div>

${attendees.length > 0 ? `
<div style="background: white;border-radius:8px;padding:24px;border:1px solid #e5e7e9;">
  <h3 style="margin:0 0 12px;font-size:15px;color:#374151;">Attendees </h3>
  <table style="width:100%;border-collapse: collapse;font-size:14px;">
    <thead>
      <tr style="background: #f3f4f6;">
        <th style="padding:8px 12px;text-align: left;color:#6b7280;font-weight:500;">Name
        <th style="padding:8px 12px;text-align: left;color:#6b7280;font-weight:500;">Email
      </tr>
    </thead>
    <tbody>${attendeeRows}</tbody>
  </table>
</div>` : ""}

<p style="color:#9ca3af;font-size:12px;text-align: center;margin-top:24px;">
  EventHub Admin Portal Booking Management
</p>
</div>

`,
};

await transporter.sendMail(mailOptions);
console.log(` Booking accepted email sent to ${adminEmail} `);
}

const express = require("express");

```

```

const cors = require("cors");
const pool = require("./db");
require("dotenv").config();

const authRoutes = require("./routes/auth");
const eventRoutes = require("./routes/events");
const bookingRoutes = require("./routes/bookings");
const adminRoutes = require("./routes/admin");

const app = express();
const PORT = process.env.PORT || 4000;

// Middleware
app.use(cors({ origin: ["http://localhost:5173", "http://localhost:8080", "http://localhost:3000"] }));
app.use(express.json());

// Routes
app.use("/api/auth", authRoutes);
app.use("/api/events", eventRoutes);
app.use("/api/bookings", bookingRoutes);
app.use("/api/admin", adminRoutes);

// Health check
app.get("/api/health", (req, res) => {
  res.json({ status: "ok", message: "EventHub API is running" });
});

// Test DB connection then start server
pool.getConnection()
  .then((conn) => {
    conn.release();
    console.log(" MySQL connected successfully");
    app.listen(PORT, () => {
      console.log(` Server running at http://localhost:${PORT} `);
      console.log(` API endpoints: `);
      console.log(` POST http://localhost:${PORT}/api/auth/register `);
      console.log(` POST http://localhost:${PORT}/api/auth/login `);
      console.log(` GET http://localhost:${PORT}/api/events `);
      console.log(` GET http://localhost:${PORT}/api/admin/users `);
    });
  });

```

```
    });  
  })  
  .catch((err) => {  
    console.error("    ■MySQL■connection■failed:", err.message);  
    console.error("Make■sure■MySQL■is■running■and■credentials■are■correct.");  
    process.exit(1);  
  });
```

REFERENCES

[1] Meta Platforms Inc. (2024). React: A JavaScript Library for Building User Interfaces, React Official Documentation, 1(1), pp. 1–10.

<https://react.dev/>

[2] OpenJS Foundation (2024). Node.js Runtime Environment Documentation, Node.js Official Docs, 2(1), pp. 1–12.

<https://nodejs.org/en/docs>

[3] Oracle Corporation (2024). MySQL Relational Database Management System, MySQL Official Documentation, 3(2), pp. 1–15.

<https://dev.mysql.com/doc/>

[4] BookMyShow Team (2023). Online Ticket Booking System Architecture and Implementation, BookMyShow Technical Report, 45–52.

<https://tech.bookmyshow.com/>

[5] Eventbrite Inc. (2023). Event Management and Ticketing Platform System Design, Eventbrite Engineering Journal, 4(3), pp. 30–38.

<https://www.eventbrite.com/engineering/>